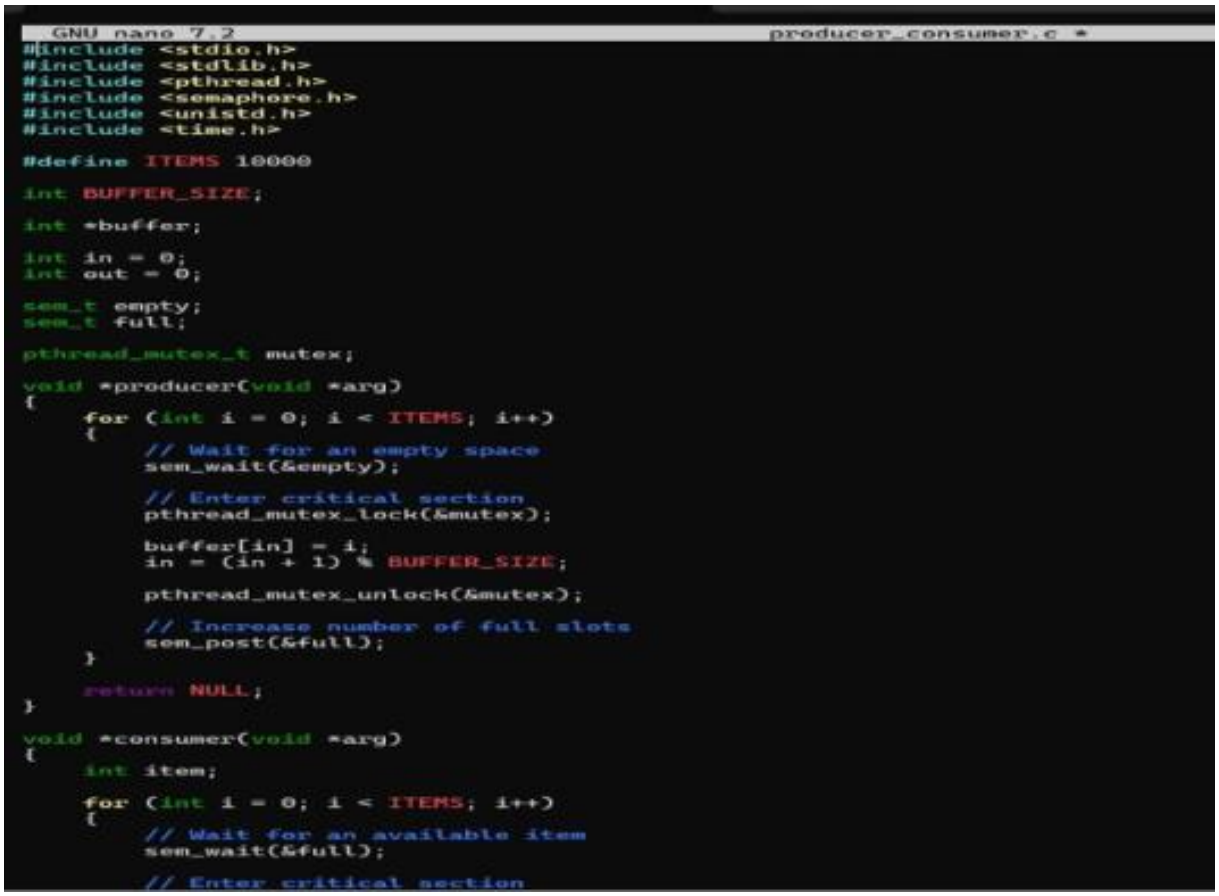
PRACTICALWEEK12

TASK1: Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.



```
  GNU nano 7.2                                    producer_consumer.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define ITEMS 10000

int BUFFER_SIZE;

int *buffer;

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 0; i < ITEMS; i++)
    {
        // Wait for an empty space
        sem_wait(&empty);

        // Enter critical section
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        // Increase number of full slots
        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg)
{
    int item;

    for (int i = 0; i < ITEMS; i++)
    {
        // Wait for an available item
        sem_wait(&full);

        // Enter critical section
```

```c
    double time_taken =
        (double)(end - start) / CLOCKS_PER_SEC;

    printf("\nProducer-Consumer completed successfully.\n");
    printf("Buffer size: %d\n", BUFFER_SIZE);
    printf("Items produced: %d\n", ITEMS);
    printf("Items consumed: %d\n", ITEMS);
    printf("Execution time: %f seconds\n", time_taken);

    if (time_taken > 0)
    {
        printf("Throughput: %.2f items/second\n",
               ITEMS / time_taken);
    }

    // Destroy synchronization objects
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    free(buffer);

    return 0;
}
```

```
manikanta@Hemu: ~/
manikanta@Hemu:~/Practica12$ mkdir Practical12
manikanta@Hemu:~/Practica12$ cd Practical12
manikanta@Hemu:~/Practica12$ nano producer_consumer.c
manikanta@Hemu:~/Practica12$ gcc producer_consumer.c -o producer_consumer
-pthread
manikanta@Hemu:~/Practica12$ ./producer_consumer
Enter buffer size: 5

Producer-Consumer completed successfully.
Buffer size: 5
Items produced: 10000
Items consumed: 10000
Execution time: 0.088965 seconds
Throughput: 112403.75 items/second

manikanta@Hemu:~/Practica12$ ./producer_consumer
Enter buffer size: 2

Producer-Consumer completed successfully.
Buffer size: 2
Items produced: 10000
Items consumed: 10000
Execution time: 0.195671 seconds
Throughput: 51106.19 items/second

manikanta@Hemu:~/Practica12$
```

TASK2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```
  GNU nano 7.2                                  deadlock.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1_function(void *arg)
{
    printf("Thread 1: Trying to lock Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: Locked Resource 1\n");

    sleep(1);

    printf("Thread 1: Trying to lock Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Locked Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2_function(void *arg)
{
    printf("Thread 2: Trying to lock Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Locked Resource 2\n");

    sleep(1);

    printf("Thread 2: Trying to lock Resource 1\n");
    pthread_mutex_lock(&resource1);
```

```
    printf("Thread 2: Trying to lock Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Locked Resource 1\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

int main()
{
    pthread_t thread1, thread2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&thread1, NULL,
                   thread1_function, NULL);

    pthread_create(&thread2, NULL,
                   thread2_function, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("Program completed.\n");

    return 0;
}
```

```
  GNU nano 7.2                                  deadlock_prevent
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1_function(void *arg)
{
    printf("Thread 1: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: Resource 1 locked\n");

    sleep(1);

    printf("Thread 1: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Resource 2 locked\n");

    printf("Thread 1: Using both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 1: Resources released\n");

    return NULL;
}

void *thread2_function(void *arg)
{
    // IMPORTANT:
    // Thread 2 also locks Resource 1 first

    printf("Thread 2: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);
```

```
  GNU nano 7.2                                  deadlock_prevention.
    printf("Thread 2: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Resource 2 locked\n");

    printf("Thread 2: Using both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 2: Resources released\n");

    return NULL;
}

int main()
{
    pthread_t thread1, thread2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&thread1, NULL,
                   thread1_function, NULL);

    pthread_create(&thread2, NULL,
                   thread2_function, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("\nProgram completed successfully.\n");

    return 0;
}
```

```
Throughput: 51106.19 items/second
manikanta@Hemu:~/Practical12$ nano deadlock.c
manikanta@Hemu:~/Practical12$ gcc deadlock.c -o deadlock -
pthread
manikanta@Hemu:~/Practical12$ ./deadlock
Thread 1: Trying to lock Resource 1
Thread 1: Locked Resource 1
Thread 2: Trying to lock Resource 2
Thread 2: Locked Resource 2
Thread 1: Trying to lock Resource 2
Thread 2: Trying to lock Resource 1
^C
manikanta@Hemu:~/Practical12$ nano deadlock_prevenenhock_prevention.c
manikanta@Hemu:~/Practical12$ gcc deadlock_prevention.c -o
deadlock_prevention -pthread
manikanta@Hemu:~/Practical12$ ./deadlock_prevention
Thread 1: Locking Resource 1
Thread 1: Resource 1 locked
Thread 2: Locking Resource 1
Thread 1: Locking Resource 2
Thread 1: Resource 2 locked
Thread 1: Using both resources
Thread 1: Resources released
Thread 2: Resource 1 locked
Thread 2: Locking Resource 2
Thread 2: Resource 2 locked
Thread 2: Using both resources
Thread 2: Resources released

Program completed successfully.
manikanta@Hemu:~/Practical12$
```